Supplementary Material for

ProbSPARQL: Querying Knowledge Graphs with Multi-dimensional, Uncertain Numeric Data

Table of Contents

Jingcheng: To improve

S.1 Scope and Relationship to the Main Paper

The main paper introduces ProbSPARQL as a SPARQL-compatible query-layer extension for RDF knowledge graphs in which uncertain numeric measurements are represented by random-variable nodes linked to distribution-valued RDF literals. The paper focuses on the circular-factory inspection scenario and reports the principal evaluation results for probabilistic filtering, uncertainty propagation, divergence-based matching, and datatype extensibility.

This supplementary material provides supporting formal, implementation, and experimental details for ProbSPARQL. It specifies the probabilistic foundations, well-formedness constraints for probabilistic RDF literal datatypes, datatype-specific semantics for distribution-valued and deterministic-valued operators, evaluation procedures for scalar divergence functions and hypothesis-test-based divergence matching, complete query workloads, benchmark construction procedures, extended evaluation protocols, multi-dimensional Gaussian mixture model (GMM) experiments, algebraic properties of the ProbSPARQL evaluation model, and reproducibility details. It is intended as supporting material and does not introduce additional claims beyond those stated in the main paper.

S.2 Probabilistic Foundations

This section fixes the probabilistic notation used throughout the supplementary material. Following the main paper, ProbSPARQL treats random variables extensionally, i.e., up to their induced probability distribution. We therefore represent an uncertain numeric measurement as a pair $X = (D, P)$, where $D \subseteq \mathbb{R}^d$ is a measurable value domain equipped with the Borel σ-algebra $\mathcal{B}(D)$ and P is a probability measure over $(D, \mathcal{B}(D))$.

Distributional equivalence. Following the main paper, two compatible random variables $X = (D, P)$ and $X' = (D, P')$ are distributionally equivalent, written $X \equiv X'$, iff their probability measures coincide. Equivalently, in the measure-theoretic notation used in this report, this means that $P(A) = P'(A)$ for every measurable set $A \in \mathcal{B}(D)$. This notion is semantic rather than lexical: two probabilistic literals may be distributionally equivalent even if their lexical encodings are not identical.

Definition S.1 (Kullback–Leibler divergence). *Let P and Q be probability measures over $(D, \mathcal{B}(D))$. If P is absolutely continuous with respect to Q, written $P \ll Q$, the Kullback–Leibler divergence is*

$$\mathrm{KL}(P\|Q) = \int_D \log\left(\frac{dP}{dQ}\right) dP.$$

If $P \not\ll Q$, we set $\mathrm{KL}(P\|Q) = \infty$. When P and Q admit densities p and q with respect to a common reference measure λ, this becomes

$$\mathrm{KL}(P\|Q) = \int_D p(x) \log \frac{p(x)}{q(x)}\, d\lambda(x).$$

In the continuous case with Lebesgue densities, $d\lambda(x)$ is written as dx. KL divergence is non-negative, equals zero iff $P = Q$, and is generally asymmetric. We use natural logarithms throughout.

Definition S.2 (Jensen–Shannon divergence). *Let P and Q be probability distributions over the same measurable domain and let $M = \frac{1}{2}(P + Q)$. The Jensen–Shannon divergence is*

$$\mathrm{JSD}(P, Q) = \frac{1}{2}\mathrm{KL}(P\|M) + \frac{1}{2}\mathrm{KL}(Q\|M).$$

It is symmetric, satisfies $\mathrm{JSD}(P, Q) = 0$ iff $P = Q$, and is bounded by $0 \leq \mathrm{JSD}(P, Q) \leq \log 2$ when natural logarithms are used. Moreover, $\sqrt{\mathrm{JSD}}$ is a metric on probability distributions.

Definition S.3 (Compatibility-oriented divergence test). *Let $X = (D, P)$ and $X' = (D, P')$ be compatible random variables, i.e., random variables over the same measurable domain, and let Δ be a divergence measure satisfying $\Delta(X, X') = 0$ iff $X \equiv X'$. Given a tolerance $\epsilon \geq 0$ and a significance level $\alpha \in (0, 1)$, the compatibility-oriented divergence test evaluates*

$$H_0 : \Delta(X, X') \leq \epsilon \qquad \textit{against} \qquad H_1 : \Delta(X, X') > \epsilon.$$

The Boolean result is true when H_0 cannot be rejected under the configured decision strategy.

In ProbSPARQL, this test is exposed through `prob:divergenceTest` and through the `DIVJOIN` operator. When the configured backend implements a statistical test with a formal error guarantee, α has its standard significance-level interpretation. For approximate or cascade-based decision strategies, α is used as a decision-control parameter, and the empirical error behavior is evaluated separately.

Scalar divergence versus Boolean compatibility testing. ProbSPARQL distinguishes deterministic-valued divergence functions from compatibility-oriented Boolean predicates. For compatible distribution-valued expressions, the function `prob:jsd`(E_1, E_2) returns a scalar divergence estimate and is used for anomaly-detection filters such as `prob:jsd`$(E_1, E_2) > \tau$. In contrast, `prob:divergenceTest`$(E_1, E_2, \epsilon, \alpha)$ does not return a divergence value; it returns the Boolean compatibility decision defined above. This distinction separates score-based validation queries, such as anomaly detection, from compatibility-based matching queries, such as component pairing with `DIVJOIN`.

S.3 Probabilistic Literal Datatypes

ProbSPARQL represents uncertain numeric measurements by random-variable nodes that separate the measurement domain from the encoded probability distribution. The distribution itself is stored as a probabilistic RDF literal, while the random-variable node provides the graph-level connection to the measured entity, domain, unit, and provenance information. Each probabilistic datatype is defined by a lexical space, a value space, a partial interpretation function from lexical forms to probabilistic values, and datatype-specific well-formedness constraints. Malformed lexical forms, inconsistent dimensionalities, unsupported covariance encodings, or invalid parameter values cause the interpretation to be undefined.

S.3.1 General Datatype Interpretation Scheme

For a probabilistic datatype τ, let L_τ be its lexical space and V_τ its value space. The datatype interpretation is a partial function

$$[\![\cdot]\!]_\tau : L_\tau \rightharpoonup V_\tau.$$

A probabilistic literal ℓ with datatype τ is well-formed iff its lexical form belongs to the domain of $[\![\cdot]\!]_\tau$. Operator evaluation is defined only for well-formed literals whose datatype, domain, dimensionality, and operator-specific compatibility conditions are satisfied.

S.3.2 Gaussian Mixture Model Literals (`uq:gmmLiteral`)

A `uq:gmmLiteral` encodes a Gaussian mixture model with K components and Euclidean dimension d:

$$p(x) = \sum_{k=1}^{K} w_k \, \mathcal{N}(x; \mu_k, \Sigma_k),$$

where $x \in \mathbb{R}^d$, w_k are non-negative mixture weights, $\sum_{k=1}^{K} w_k = 1$, $\mu_k \in \mathbb{R}^d$, and Σ_k is a valid covariance matrix. The literal specifies a Gaussian mixture density on $\mathbb{R}^d$. Physical domain annotations attached to the corresponding random-variable node, such as non-negative length domains, are used for compatibility checking and unit/domain metadata; they do not by themselves change the parametric GMM encoding.

A JSON lexical form is a valid `uq:gmmLiteral` iff it contains the following mandatory fields and satisfies the listed constraints:

- `n_components`: a positive integer K.
- `dimensions`: a positive integer d.
- `covariance_type`: one of `"full"`, `"diag"`, or `"spherical"`.
- `weights`: an array of K non-negative real numbers summing to one up to the configured numerical tolerance.

- `means`: a $K \times d$ array of real-valued component means.
- `covariances`: covariance parameters whose shape is determined by `covariance_type`. For `"full"`, each component has a symmetric positive-definite $d \times d$ covariance matrix. For `"diag"`, each component has a length-d vector of strictly positive diagonal entries. For `"spherical"`, each component has a strictly positive scalar variance.

S.3.3 Dirichlet Literals (`uq:dirichletLiteral`)

A `uq:dirichletLiteral` encodes a compositional random vector over the probability simplex

$$\Delta^{K-1} = \left\{ x \in \mathbb{R}^K : x_i \geq 0,\ \sum_{i=1}^{K} x_i = 1 \right\}.$$

It is multi-dimensional in the sense that it represents a random vector with K components, but its support is the $(K-1)$-dimensional simplex embedded in $\mathbb{R}^K$ rather than an unconstrained Euclidean domain. Its lexical form is a JSON object with an `alphas` field. For example:

```
{ "alphas": [1.5, 2.0, 0.5] }
```

The literal is well-formed iff the array length is $K \geq 2$ and each concentration parameter satisfies $\alpha_i > 0$.

S.3.4 Histogram Literals (`uq:histogramLiteral`)

A `uq:histogramLiteral` provides a non-parametric representation for empirical distributions over rectangular bin partitions. Its lexical form contains a `dimensions` field, a list of bin-boundary arrays, one for each dimension, and a tensor of bin masses. Histogram weights are interpreted as probability masses assigned to bins. For density evaluation, the distribution is treated as piecewise uniform inside each bin.

For example, the following lexical form encodes a two-dimensional histogram:

```
{
  "dimensions": 2,
  "bins": [
    [0.0, 10.0, 20.0],
    [0.0, 5.0, 10.0]
  ],
  "weights": [
    [0.10, 0.20],
    [0.30, 0.40]
  ]
}
```

For dimensions $j = 1, \ldots, d$, let

$$B^{(j)} = (b_0^{(j)}, \ldots, b_{n_j}^{(j)})$$

be the bin-boundary array for dimension j. A histogram literal is well-formed iff each $B^{(j)}$ is strictly increasing, the weight tensor has shape $(n_1, \ldots, n_d)$, all tensor entries are non-negative, and the tensor entries sum to one up to the configured numerical tolerance.

The represented distribution is piecewise uniform over axis-aligned hyperrectangular bins. For a bin

$$C_{i_1,\ldots,i_d} = \prod_{j=1}^{d} [b_{i_j}^{(j)}, b_{i_j+1}^{(j)}),$$

with probability mass $w_{i_1,\ldots,i_d}$, the density inside the bin is

$$\frac{w_{i_1,\ldots,i_d}}{\prod_{j=1}^{d} \left(b_{i_j+1}^{(j)} - b_{i_j}^{(j)}\right)}.$$

S.4 Datatype-Specific Operator Semantics

This section gives the generic semantics of the main ProbSPARQL operators and specifies datatype-specific behavior for the probabilistic datatypes used in the prototype. ProbSPARQL operators are partial. They are defined only when their operands are well-formed probabilistic literals or ordinary RDF literals of the expected type, and when datatype-specific dimensionality and compatibility requirements are satisfied. When an operator is undefined, expression evaluation yields an error following SPARQL's error propagation rules for expression and filter evaluation.

S.4.1 Unary Distribution-Valued Transformations

Let $X = (D, P)$ be a random variable over $D \subseteq \mathbb{R}^d$.

Shift. `prob:shift`(X, v) returns the distribution of $Y = X + v$, where v has the same dimensionality as X. For a GMM, this shifts every component mean by v and leaves mixture weights and covariances unchanged.

Scale. `prob:scale`(X, λ) returns the distribution of $Y = \lambda X$. For $\lambda \neq 0$, the transformed density is

$$p_Y(y) = |\lambda|^{-d} p_X(y/\lambda).$$

For a GMM, component means are multiplied by λ and covariances by λ^2. For $\lambda = 0$, the result is degenerate and is undefined unless the backend supports degenerate point-mass distributions.

Linear transformation. `prob:linearTransform`(X, A, b) returns the distribution of $Y = AX + b$, where the matrix A and vector b have compatible dimensions. For a GMM, each component is transformed as

$$\mu'_k = A\mu_k + b, \qquad \Sigma'_k = A\Sigma_k A^\top.$$

If the transformation produces singular covariance matrices and the target datatype requires positive-definite covariances, the operator is undefined unless the backend supports degenerate Gaussian components. If an initially diagonal or spherical GMM becomes full-covariance after transformation, the result is serialized using the supported covariance representation selected by the backend.

Marginalization. `prob:marginal`(X, S) returns the marginal distribution over a non-empty subset of dimensions $S \subseteq \{1, \ldots, d\}$. For a GMM, this selects the corresponding entries of each component mean and the corresponding covariance submatrix.

S.4.2 Binary Distribution-Valued Operators

Operators that combine multiple random variables require either an explicit joint distribution or an explicit independence assumption. When only marginal distributions are available, the implementation uses independent-product semantics only for operators whose backend explicitly declares this assumption.

Joint. `prob:joint`(X_1, X_2) constructs a joint random variable over $D_1 \times D_2$. If no joint distribution is supplied, the operator is defined only under an independence assumption and returns the product measure $P_1 \otimes P_2$. For independent GMMs, the resulting mixture has pairwise component combinations, concatenated means, block-diagonal covariance matrices, and component weights given by products of the original mixture weights.

Mixture. `prob:mix`(X_1, X_2, w) returns the mixture distribution $wP_1 + (1-w)P_2$ for compatible domains and $w \in [0, 1]$. For GMM operands with the same Euclidean dimension, this corresponds to concatenating the component sets and rescaling their weights by w and $1 - w$.

Convolution. `prob:convolve`(X_1, X_2) returns the distribution of $Y = X_1 + X_2$ and is defined for operands over compatible Euclidean domains. Under independence and for densities p_1, p_2, the density is

$$(p_1 * p_2)(y) = \int p_1(y - u)p_2(u)\, du.$$

For independent GMMs, convolution yields another GMM whose components are all pairwise sums of component means and covariances, with component weights given by products of the original mixture weights.

Multiplication. `prob:multiply`(X_1, X_2) returns the distribution of $Y = X_1 X_2$ for scalar operands. This is the form used in the motor-performance query, where uncertain power is derived from uncertain speed and torque. For vector-valued operands, the operator is defined only if the backend explicitly declares an element-wise, inner-product, or other product semantics. Since the exact product distribution is generally not closed under GMMs, the implementation may use sampling or moment-matching depending on the datatype backend.

Fusion. `prob:fuse`(X_1, X_2) combines two compatible distributions using a normalized product of densities:

$$p_{\text{fuse}}(x) = \frac{p_1(x)p_2(x)}{\int_D p_1(u)p_2(u)\,du}.$$

The operator is defined only when the normalization constant is finite and nonzero. This operator should not be confused with `prob:multiply`, which multiplies random-variable values rather than densities.

S.4.3 Deterministic-Valued Evaluation Operators

Mean and standard deviation. `prob:mean` returns $\mathbb{E}[X]$. For a GMM,

$$\mathbb{E}[X] = \sum_{k=1}^{K} w_k \mu_k.$$

`prob:std` returns the standard deviation for scalar variables. For multivariate variables, it returns the component-wise standard deviation vector, i.e., the square roots of the diagonal entries of the covariance matrix, unless the backend exposes a richer covariance summary through a separate operator.

PDF, log-PDF, and CDF. `prob:pdf` and `prob:logpdf` evaluate the density or log-density at a point of matching dimensionality. `prob:cdf` evaluates the cumulative distribution function. For multivariate operands, the evaluation point must be a vector $t = (t_1, \ldots, t_d)$ of matching dimensionality, and the CDF is interpreted as

$$F_X(t) = P(X_1 \le t_1, \ldots, X_d \le t_d).$$

Operators with one-dimensional signatures, such as scalar quantiles, are not applied directly to multivariate operands unless composed with `prob:marginal` or `prob:linearTransform`.

Quantile, MAP, and mode count. `prob:quantile` is defined for one-dimensional distributions. `prob:map` returns a maximum-a-posteriori point estimate when supported by the datatype backend. `prob:modeCount` returns the number of modes when this can be computed exactly or approximated by the backend.

Jensen–Shannon divergence. `prob:jsd`(X_1, X_2) returns a deterministic scalar divergence estimate for compatible distributions. Closed-form evaluation is generally unavailable for GMMs and other complex distribution families, so the implementation uses the numerical approximation strategies described in Section S.5.

S.5 Divergence Functions and Hypothesis-Test-Based Matching

This section describes how ProbSPARQL evaluates scalar divergence functions and Boolean compatibility decisions. The distinction follows Section S.2: `prob:jsd` returns a scalar divergence estimate, whereas `prob:divergenceTest` and `DIVJOIN` return or apply a Boolean compatibility decision.

S.5.1 Scalar Divergence Function

The deterministic-valued function `prob:jsd`(E_1, E_2) evaluates the Jensen–Shannon divergence between two compatible distribution-valued expressions. It is used when a query needs a scalar divergence value, for example in anomaly detection:

```
FILTER(prob:jsd(?d1, ?d2) > 0.2).
```

For distribution families without a closed-form JSD, the runtime operator uses numerical or sampling-based approximation. In the evaluation, these estimates are compared against high-sample reference estimates in order to report accuracy metrics such as MAE and binary-decision quality.

S.5.2 Boolean Compatibility Predicate

The Boolean predicate `prob:divergenceTest`$(E_1, E_2, \epsilon, \alpha)$ implements the divergence test from Definition S.3. It returns true when the configured decision strategy cannot reject the hypothesis that the divergence between the two operand distributions is at most ϵ.

The `DIVJOIN` operator uses the same compatibility decision as an optimizer-visible join condition. In a divergence join, candidate solution mappings are retained only when the ordinary mapping-compatibility conditions are satisfied and the configured divergence decision returns true for the designated distribution-valued variables.

The parameter α has its standard significance-level interpretation when the configured backend implements a statistical test with a formal error guarantee. For approximate or cascade-based strategies, α is used as a decision-control parameter, and empirical error behavior is evaluated experimentally.

S.5.3 Decision Strategies

The prototype supports multiple strategies for evaluating divergence-based decisions. These strategies trade off latency, estimation accuracy, and binary decision quality.

- **Fixed-budget Monte Carlo.** A fixed number of samples is drawn to estimate the divergence. The resulting estimate is then compared with the tolerance ϵ to obtain a binary decision.
- **Stratified Monte Carlo.** Stratified sampling reduces estimator variance by sampling from mixture components or strata in a controlled manner. This strategy is useful for Gaussian mixture operands, where component structure can be exploited during sampling.
- **Sequential probability ratio testing (SPRT).** SPRT evaluates the binary decision sequentially and may terminate early when sufficient evidence has accumulated for accepting or rejecting the compatibility decision.
- **Deterministic bound.** A deterministic lower bound on the divergence provides a cheap rejection test. If the lower bound already exceeds the tolerance, the candidate can be rejected without running a full sampling-based estimator. If the bound is inconclusive, a second-stage strategy is required.
- **Adaptive-Cascade.** The adaptive cascade combines the previous strategies. It first applies the deterministic bound to reject clearly incompatible candidates. For unresolved cases, it applies SPRT to obtain an early binary decision when possible. If the pair remains ambiguous near the decision boundary, the strategy falls back to a fixed-budget estimator, such as stratified Monte Carlo.

The main paper reports the latency–accuracy trade-off of these strategies for hard near-threshold workloads and mixed workloads. Additional construction details for these workloads are provided in Section S.7.

S.6 Complete ProbSPARQL Query Workloads

This section gives complete query templates for the four motivating use cases. For readability, prefix declarations are omitted from individual listings. The queries use the same prefixes as the main paper, including `ag:` for the angle-grinder vocabulary, `im:` for inspection measurements, `om:` for units of measurement, and `uq:` for probabilistic random-variable and literal vocabulary.

S.6.1 R1: Threshold-Based Retrieval

```
SELECT ?gear ?lenchar ?prob WHERE {
  ?gear a ag:CrownGear ;    im:hasLengthCharacteristic ?lenchar .
  ?lenchar im:isMeasuredBy ?measurement .
  ?measurement om:hasUnit om:millimetre ;   uq:representedBy ?rv .
```

```
  ?rv uq:hasDomain uq:nonNegativeDouble ;   uq:hasDistribution ?dist .
  BIND(prob:cdf(?dist, 9.8) AS ?prob)
  FILTER(?prob >= 0.9)
}
```

Listing S.1. R1: Threshold-based retrieval using a probabilistic CDF filter.

S.6.2 R2: Anomaly Detection with Scalar Divergence

```
SELECT ?gear ?div WHERE {
  ?gear a ag:CrownGear ;    im:hasLengthCharacteristic ?char .
  ?char im:isMeasuredBy ?ctMeasurement ;    im:isMeasuredBy ?slMeasurement .
  ?ctMeasurement a im:ComputedTomographyMeasurement ;   uq:representedBy ?rvCT .
  ?slMeasurement a im:StructuredLightMeasurement ;  uq:representedBy ?rvSL .
  ?rvCT uq:hasDistribution ?distCT .
  ?rvSL uq:hasDistribution ?distSL .
  BIND(prob:jsd(?distCT, ?distSL) AS ?div)
  FILTER(?div > 0.2)
}
```

Listing S.2. R2: Anomaly detection using scalar Jensen–Shannon divergence.

S.6.3 R3: Motor Performance Evaluation by Uncertainty Propagation

```
SELECT ?motor ?powerDist WHERE {
  ?motor a ag:Motor ;
         ag:hasSpeedMeasurement ?speedMeasurement ;
         ag:hasTorqueMeasurement ?torqueMeasurement .
  ?speedMeasurement uq:representedBy ?speedRV .
  ?torqueMeasurement uq:representedBy ?torqueRV .
  ?speedRV uq:hasDistribution ?speedDist .
  ?torqueRV uq:hasDistribution ?torqueDist .
  BIND(prob:multiply(?speedDist, ?torqueDist) AS ?powerDist)
}
```

Listing S.3. R3: Uncertainty propagation from speed and torque to power.

The R3 listing assumes that speed and torque distributions have already been converted to compatible physical units before applying `prob:multiply`.

S.6.4 R4: Component Pairing with Divergence Join

```
SELECT ?driveGear ?spindleGear WHERE {
  {
    ?driveGear a ag:DriveGear ; im:hasLengthCharacteristic ?lencharA .
    ?lencharA im:isMeasuredBy ?measA .
    ?measA om:hasUnit om:millimetre ;   uq:representedBy ?rvA .
    ?rvA uq:hasDomain uq:nonNegativeDouble ;    uq:hasDistribution ?distA .
  }
  DIVJOIN(?distA, ?distB, 0.1, 0.05)
  {
    ?spindleGear a ag:SpindleGear ; im:hasLengthCharacteristic ?lencharB .
    ?lencharB im:isMeasuredBy ?measB .
    ?measB om:hasUnit om:millimetre ;   uq:representedBy ?rvB .
```

```
    ?rvB uq:hasDomain uq:nonNegativeDouble ;    uq:hasDistribution ?distB .
  }
}
```

Listing S.4. R4: Divergence join for compatibility-oriented component pairing with epsilon = 0.1 and alpha = 0.05.

S.7 Benchmark Construction

S.7.1 Generated Circular-Factory Knowledge Graph

The benchmark graphs mirror the circular-factory schema used in the main paper, including angle-grinder instances, component instances, measurable characteristics, measurement records, random-variable nodes, unit annotations, scalar summaries, and probabilistic RDF literals. The benchmark family is constructed for controlled experiments and ranges from 10 angle-grinder instances with 3,060 triples to 5,000 angle-grinder instances with 1,525,010 triples. Different experiments use different members of this benchmark family: distribution-complexity experiments fix the graph size, whereas graph-size scalability experiments vary the number of generated angle-grinder instances. Table S.1 summarizes the benchmark configurations used for these controlled experiments.

The generated graphs preserve the graph traversal structure of the circular-factory query layer while making distribution parameters, filter selectivity, and divergence difficulty independently controllable. They are therefore used as reproducible workloads for controlled experiments, rather than as a claim to reproduce the full empirical SFB 1574 knowledge graph.

Table S.1. Size and composition of the controlled circular-factory benchmark graphs. P = products, C = components, Char. = measurable characteristics, Meas. = measurement records, RVs = random-variable nodes.

Graph	P	C	Char.	Meas.	RVs	Triples
S1	10	30	80	240	240	3,078
S2	100	300	800	2,400	2,400	30,708
S3	1,000	3,000	8,000	24,000	24,000	307,008
S4	5,000	15,000	40,000	120,000	120,000	1,535,008

S.7.2 Distribution Generation

For scalar inspection quantities modelled by one-dimensional GMMs, the number of mixture components is varied over $K \in \{1, 3, 5, 10\}$. Mixture weights are sampled as positive values and normalized to sum to one. Component means are drawn from numeric ranges corresponding to the inspection quantities used in

the query workloads, and variance parameters are sampled from strictly positive intervals to ensure valid Gaussian components and avoid degenerate zero-variance distributions. Random seeds are fixed in the benchmark scripts to make graph generation and query workload construction reproducible.

The generated distributions support controlled selectivity for threshold filters and controlled decision difficulty for divergence workloads. For threshold-filter workloads, selectivity is controlled at workload-construction time by selecting query thresholds that produce target pass fractions on the generated graph. For divergence workloads, a pool of distribution pairs is generated and later grouped according to reference JSD-based decision difficulty, as specified in Section S.7.3.

Distribution-complexity experiments vary K while keeping the graph size fixed. Graph-size scalability experiments fix $K = 3$ and vary the number of generated angle-grinder instances. This separation avoids conflating the cost of probabilistic literal evaluation with the cost of RDF graph matching over larger benchmark graphs.

S.7.3 Divergence-Decision Workloads

Candidate pairs for divergence-decision evaluation are grouped by decision difficulty. For each candidate pair i, we estimate a reference Jensen–Shannon divergence (JSD) value Δ_i^{ref} using 10^6 Monte Carlo samples. These reference estimates are used only for workload construction and evaluation, not during ordinary query execution. Unless otherwise stated, the compatibility tolerance is set to $\epsilon = 0.3$. We define the decision margin of pair i as

$$m_i = \left| \Delta_i^{\mathrm{ref}} - \epsilon \right| .$$

Small margins correspond to near-threshold pairs whose compatibility decision is sensitive to estimation error, whereas large margins correspond to pairs that are clearly below or above the tolerance threshold.

We construct four workload regimes from the margin distribution. The Easy workload samples pairs from the highest-margin region, where reference divergences are far from the decision boundary. The Medium workload samples pairs around the median margin. The Hard workload samples pairs from the lowest-margin region, where reference divergences lie close to the tolerance threshold and the decision is most ambiguous. The Mixed workload combines equal numbers of pairs from the Easy, Medium, and Hard regimes to approximate heterogeneous query conditions with both far-from-threshold and near-threshold pairs.

Each workload contains $N = 2400$ candidate pairs sampled from the fixed-size S3 benchmark graph with 1,000 generated angle-grinder instances. The resulting workloads are used to compare divergence-decision strategies, including fixed-budget Monte Carlo, stratified Monte Carlo, sequential probability ratio testing (SPRT), a deterministic lower-bound strategy, and Adaptive-Cascade. Because the graph size and candidate-pair count are fixed, these measurements isolate the per-candidate-pair cost and accuracy of divergence decisions from graph-size scalability effects.

S.7.4 Execution Environment

All Java-based experiments are run on the same hardware and software stack: an Apple MacBook Pro with an Apple M4 Pro chip, 48 GB RAM, macOS 15.7.4, Java 21, and Apache Jena 6.0.0-SNAPSHOT. The extended ARQ module is built against Apache Jena 6.0.0-SNAPSHOT. Selected Fuseki and TDB2 modules use official Apache Jena 5.6.0 dependencies in the project setup. The exact build configuration is included in the released artifact. Queries are executed through the extended Jena ARQ engine. When endpoint execution is evaluated, the same engine is exposed through Apache Jena Fuseki. Unless otherwise stated, each reported latency is the median of 10 runs after 3 warm-up iterations. Warm-up runs are excluded from the reported measurements. Application-layer post-processing baselines are executed on the same machine.

S.8 Extended Evaluation Protocols and Result Tables

This section provides extended protocols and result tables supporting the evaluation in the main paper. This section covers four controlled evaluation dimensions: distribution-complexity overhead, knowledge-graph size scalability, filter-pushdown behavior, and divergence-decision strategies. It also reports an applicability check on a real circular-factory knowledge-graph fragment. Graph-size scaling and distribution-complexity scaling are evaluated separately. Graph-size scaling varies the number of ontology-conformant angle-grinder instances, while distribution-complexity scaling fixes the benchmark size and varies the number of mixture components K. The protocols below specify how reference estimates, accuracy metrics, benchmark configurations, and latency measurements are obtained.

Unless otherwise stated, latency values are reported as medians over repeated runs after warm-up, following the evaluation protocol in Section S.7.4.

S.8.1 Reference Divergence Estimates

For divergence-evaluation experiments, reference JSD values are estimated with 10^6 Monte Carlo samples per candidate pair. These values are used as empirical references only for workload construction and evaluation.

For numerical divergence estimates, the reference values are used to compute mean absolute error (MAE):

$$\mathrm{MAE} = \frac{1}{N} \sum_{i=1}^{N} \left| \widehat{\Delta}_i - \Delta_i^{\mathrm{ref}} \right|,$$

where N is the number of evaluated candidate pairs, $\widehat{\Delta}_i$ is the divergence estimate produced by the evaluated strategy, and Δ_i^{ref} is the corresponding reference estimate.

Table S.2. Median latency under different GMM mixture-component counts. The K-columns report ProbSPARQL GMM latency in milliseconds. Det. denotes the corresponding deterministic, non-probabilistic query variant. Distribution comparison has no deterministic counterpart.

Query type	**Det.**	**ProbSPARQL (ms)**				**Overhead**
	(ms)	$K = 1$	$K = 3$	$K = 5$	$K = 10$	(×)
Retrieval	170.66	242.43	213.44	233.45	220.17	1.25–1.42
Probabilistic filtering	172.16	275.31	245.66	249.43	248.21	1.43–1.60
Uncertainty propagation	31.91	55.21	54.31	57.81	62.13	1.70–1.95
Distribution comparison	–	1852.80	4339.60	6490.01	10744.45	–

For binary divergence decisions, reference JSD values are converted into reference labels using the tolerance $\epsilon = 0.3$. A pair is labelled reference-compatible if $\Delta_i^{\text{ref}} \leq \epsilon$ and reference-incompatible otherwise. The reference-compatible class is used as the positive class. Predicted decisions are then compared with these reference labels to compute precision, recall, and F1 score. MAE evaluates numerical divergence estimation, whereas precision, recall, and F1 evaluate the induced binary divergence decision.

S.8.2 Distribution-Complexity Overhead

This experiment isolates the effect of distributional complexity by varying the number of GMM components K on a fixed-size benchmark graph. It is therefore separate from the graph-size scalability experiment, which varies the number of generated angle-grinder instances. For each query type, the ProbSPARQL variant is compared with its corresponding deterministic counterpart where such a counterpart exists. The deterministic counterparts use scalar summaries and ordinary SPARQL expressions while preserving the same graph-pattern structure.

Table S.2 reports the resulting median latencies and overhead ranges for retrieval, probabilistic filtering, uncertainty propagation, and distribution comparison.

As shown in Table S.2, retrieval, probabilistic filtering, and uncertainty propagation incur moderate overhead across $K \in \{1, 3, 5, 10\}$. The weak and non-monotonic dependence on K indicates that K is not the primary cost driver for these workloads at this benchmark size. This is consistent with fixed graph-pattern evaluation and probabilistic literal handling contributing a substantial part of the runtime. Distribution comparison behaves differently: its latency increases substantially with K, reflecting the higher cost of sampling-based divergence estimation.

S.8.3 Knowledge-Graph Size Scalability

Graph-size scalability is evaluated on graphs S1–S4 from Table S.1, varying the number of generated angle-grinder instances from 10 to 5,000 while fixing

Table S.3. Knowledge-graph size scalability with fixed GMM mixture-component count $K = 3$. Graph sizes are defined in Table S.1. Latencies are median execution times in milliseconds.

Graph	Det. Ret.	Prob. Ret.	Det. Filt.	Prob. Filt.	Prob. Comp.
S1	0.89	0.68	0.62	0.81	21.52
S2	0.79	1.30	1.11	1.52	191.20
S3	15.67	42.64	16.71	43.12	2,412.23
S4	170.66	213.44	172.16	245.66	4,339.60

$K = 3$. Table S.3 reports median latencies for representative deterministic and ProbSPARQL query variants. Det., Prob., Ret., Filt., and Comp. denote deterministic, ProbSPARQL, retrieval, filtering, and distribution comparison, respectively.

As shown in Table S.3, the scalability experiment characterizes growth trends with respect to RDF graph size rather than the effect of distributional complexity. Retrieval and filtering show broadly similar qualitative growth patterns for the deterministic and ProbSPARQL variants, especially on the larger benchmark graphs. At the smallest graph sizes, sub-millisecond differences should not be interpreted as systematic speed differences. Distribution comparison incurs a substantially higher absolute cost. Its scaling also depends on the number of evaluated distribution pairs rather than only on the RDF graph size.

S.8.4 Filter Pushdown and Post-Processing Baseline

The filter-pushdown experiment compares in-engine probabilistic filtering against application-layer post-processing. The in-engine strategy evaluates the CDF predicate inside the SPARQL engine before materializing and transferring the filtered results. The post-processing baseline retrieves all relevant distribution literals and applies the same CDF predicate in the application layer. Speedup is computed as post-processing latency divided by in-engine latency. Table S.4 reports the resulting median latencies and speedups.

Table S.4. Median CDF-filter latency at different pass fractions under in-engine evaluation and application-layer post-processing. Speedup is computed as post-processing latency divided by in-engine latency.

Pass fraction	In-engine (ms)	Post-processing (ms)	Speedup
0.01	521.65	1779.20	3.41×
0.05	535.29	1856.25	3.47×
0.10	610.96	1791.18	2.93×
0.30	938.36	1842.93	1.96×

Table S.5. Precision, recall, F1, MAE, and median per-candidate-pair latency of divergence-decision strategies across workload difficulties. Precision, recall, and F1 are reported as percentages. Each workload contains $N = 2400$ candidate pairs. Pairs with reference JSD not exceeding $\epsilon = 0.3$ are treated as the positive class.

Workload	Method	Prec. (%)	Rec. (%)	F1 (%)	MAE	Lat. (ms)
Easy	MC-Naive	100.00	100.00	100.00	0.000932	24.602
Easy	MC-Stratified	100.00	100.00	100.00	0.000523	23.991
Easy	SPRT	100.00	100.00	100.00	0.002203	0.260
Easy	Det-Bound	100.00	100.00	100.00	0.003713	0.009
Easy	Adaptive-Cascade	100.00	100.00	100.00	0.002681	0.124
Medium	MC-Naive	100.00	100.00	100.00	0.003717	23.927
Medium	MC-Stratified	100.00	100.00	100.00	0.001985	23.940
Medium	SPRT	100.00	100.00	100.00	0.006310	0.376
Medium	Det-Bound	100.00	100.00	100.00	0.012443	0.005
Medium	Adaptive-Cascade	100.00	100.00	100.00	0.010274	0.195
Hard	MC-Naive	96.05	97.33	96.69	0.004103	25.683
Hard	MC-Stratified	96.73	98.67	97.69	0.002125	23.337
Hard	SPRT	95.24	93.33	94.28	0.006199	4.468
Hard	Det-Bound	90.36	100.00	94.94	0.018363	0.005
Hard	Adaptive-Cascade	98.59	93.33	95.89	0.010707	3.269
Mixed	MC-Naive	100.00	98.64	99.32	0.002770	25.089
Mixed	MC-Stratified	100.00	99.32	99.66	0.001498	25.887
Mixed	SPRT	97.96	97.96	97.96	0.004767	1.810
Mixed	Det-Bound	96.69	99.32	97.99	0.011481	0.012
Mixed	Adaptive-Cascade	99.32	98.64	98.98	0.007800	1.039

As shown in Table S.4, the post-processing baseline remains nearly constant across pass fractions because all candidate distribution literals must be materialized before filtering. In-engine evaluation applies the predicate before result transfer, so it benefits most at low pass fractions. The observed speedups range from 1.96× to 3.47×.

S.8.5 Divergence-Decision Strategy Results

Divergence joins require binary decisions over candidate pairs. This experiment isolates the per-candidate-pair cost and accuracy of alternative divergence-decision strategies. Each workload contains $N = 2400$ candidate pairs sampled from the fixed-size S3 benchmark graph with 1,000 generated angle-grinder instances and uses $\epsilon = 0.3$ as the tolerance. The goal is not to identify a universally optimal strategy, but to characterize latency–accuracy trade-offs.

As shown in Table S.5, no decision strategy dominates across all metrics. MC-Stratified achieves the highest F1 score on the hard workload, but its latency remains comparable to MC-Naive. Compared with the fixed-budget Monte Carlo

Table S.6. Representative real-data tasks executed on the circular-factory KG fragment.

Task	Real-data question	ProbSPARQL feature
RMS threshold filtering	Which samples satisfy $P(\text{RMS} > 700) \geq 0.9$?	CDF-based probabilistic filtering
Replicate comparison	Which repeated measurements under the same condition are distributionally inconsistent?	JSD / divergence testing
Compatible pairing	Which samples have compatible RMS distributions?	Divergence-based join
Histogram CDF	What is the CDF value of empirical C02 roughness distributions?	Histogram-literal evaluation

variants, SPRT and Adaptive-Cascade substantially reduce latency, but their recall decreases on the hard near-threshold workload. Det-Bound is orders of magnitude faster because it evaluates a conservative lower bound rather than a full JSD estimator. For Det-Bound, MAE quantifies the deviation of the bound from the reference JSD and should not be interpreted as the error of a full divergence estimator. Its high recall and lower precision reflect a bias toward classifying pairs as reference-compatible. These results motivate exposing the divergence-decision strategy as an application-dependent configuration rather than hard-coding a single default.

S.8.6 Applicability on Real Circular-Factory Measurement Data

Before running controlled synthetic benchmarks, we validate ProbSPARQL on a real circular-factory KG fragment derived from project measurement data. The fragment contains two real-data subsets: Hardness/Temp/RMS/RS measurements and C02 roughness measurements. Both are represented using the same modelling pattern as the controlled benchmarks: samples are linked to characteristics, measurements, random-variable nodes, and probabilistic RDF literals. RMS and RS uncertainties are encoded as GMM literals, while the C02 roughness case uses histogram-based literals.

Table S.6 summarizes the representative real-data tasks. The queries cover probabilistic threshold filtering, divergence-based replicate checking, distribution-compatible pairing, and histogram CDF evaluation. This real-data check demonstrates that the datatype layer and probabilistic operators can be executed on project measurement data. Scalability and runtime behavior are evaluated separately on ontology-conformant synthetic benchmarks, where graph size, mixture complexity, and workload difficulty can be controlled.

S.8.7 Evaluation of Datatype Extensibility

We evaluate the polymorphic operator interface using three probabilistic datatype implementations: GMM, Dirichlet, and histogram literals. Using R2 as the

representative distribution-comparison query, results over a 500-entity subset agree with independent 10^6-sample reference estimates across all three datatypes. Median per-comparison latency is 0.1 ms for histograms, 1.2 ms for Dirichlet distributions, and 3.8 ms for GMMs. We further evaluate cross-type comparisons after mapping operands to a common measurable domain. Table S.7 shows that combinations involving histogram bins benefit from Det-Bound pre-filtering, while GMM–Dirichlet comparisons fall back to SPRT and reach 93.5% F1 at 5.5 ms latency. Details on cross-type routing and reference-domain mappings are provided in the supplement.

Table S.7. Cross-type distribution-comparison evaluation on controlled test cases ($\epsilon = 0.1$, 10^6 sample reference estimates). Results include precision, recall, F1 score (%), and per-comparison latency (ms).

Combination	**Pre-filter**	**Prec.**	**Rec.**	**F1**	**Lat.**
GMM vs Histogram	Det-Bound	98.2	96.8	97.5	1.8
Dirichlet vs Histogram	Det-Bound	97.1	94.9	96.0	2.3
GMM vs Dirichlet	None	94.8	92.2	93.5	5.5

S.9 Algebraic Properties and Proofs

This section records algebraic properties that are summarized in the main paper. We use the set-based solution-mapping notation of the main paper. The same equivalences extend to bag semantics when multiplicities are inherited from the corresponding ordinary joins and selections.

S.9.1 Theta-Join as Filter-After-Join

Let Θ be a finite set of atomic divergence constraints of the form $?x \sim_{\epsilon,\alpha} ?y$. For each atomic constraint $\theta = (?x \sim_{\epsilon,\alpha} ?y)$, let F_θ be the corresponding Boolean filter predicate

$$F_\theta = \texttt{prob:divergenceTest}(?x, ?y, \epsilon, \alpha).$$

For a finite set $\Theta = \{\theta_1, \ldots, \theta_m\}$, define

$$F_\Theta = F_{\theta_1} \wedge \cdots \wedge F_{\theta_m}.$$

We assume that all variables referenced by F_Θ are in scope after joining the two operands and that the divergence constraints are filter-definable by the corresponding Boolean predicates.

Proposition S.1 (Theta-join as filter-after-join). Let Ω_1 and Ω_2 be sets of solution mappings, and let Θ be a finite set of filter-definable divergence constraints whose variables are in scope over $\Omega_1 \bowtie \Omega_2$. Then

$$\Omega_1 \bowtie_\Theta \Omega_2 = \sigma_{F_\Theta}(\Omega_1 \bowtie \Omega_2).$$

Thus, the dedicated Θ-join operator does not increase expressive power in the filter-definable fragment. Its role is to expose divergence-based matching as an optimizer-visible algebraic operator.

Proof. By the definition of ordinary join, a solution mapping belongs to $\Omega_1 \bowtie \Omega_2$ iff it has the form $\mu_1 \uplus \mu_2$ for some $\mu_1 \in \Omega_1$ and $\mu_2 \in \Omega_2$ such that μ_1 and μ_2 are compatible.

By the definition of Θ-join, the same combined mapping $\mu_1 \uplus \mu_2$ belongs to $\Omega_1 \bowtie_\Theta \Omega_2$ iff, in addition to ordinary mapping compatibility, every atomic divergence constraint in Θ is satisfied by μ_1 and μ_2.

For each atomic constraint $\theta \in \Theta$, the corresponding filter predicate F_θ is constructed to be true under the combined mapping $\mu_1 \uplus \mu_2$ exactly when θ is satisfied. Hence the conjunction F_Θ is true under $\mu_1 \uplus \mu_2$ exactly when all constraints in Θ are satisfied. Therefore, selecting from the ordinary join by F_Θ retains exactly the same combined mappings as the Θ-join:

$$\mu_1 \uplus \mu_2 \in \Omega_1 \bowtie_\Theta \Omega_2 \quad \text{iff} \quad \mu_1 \uplus \mu_2 \in \sigma_{F_\Theta}(\Omega_1 \bowtie \Omega_2).$$

Thus the two expressions denote the same set of solution mappings.

S.9.2 Projection Conditions

Projection interacts with divergence constraints in the same way it interacts with ordinary SPARQL filters: a filter can only be evaluated after projection if all variables required by the filter remain in scope.

Proposition S.2 (Safe projection). Let W be a set of projected variables. If $\mathrm{var}(F_\Theta) \subseteq W$, then

$$\pi_W\big(\sigma_{F_\Theta}(\Omega)\big) = \sigma_{F_\Theta}\big(\pi_W(\Omega)\big).$$

If $\mathrm{var}(F_\Theta) \nsubseteq W$, this equivalence need not hold.

Proof. If $\mathrm{var}(F_\Theta) \subseteq W$, projection preserves every variable needed to evaluate F_Θ. Therefore, for every solution mapping μ, the truth value of F_Θ under μ is the same as its truth value under the projected mapping $\pi_W(\mu)$. Selecting before projection and selecting after projection therefore retain the same projected mappings.

If $\mathrm{var}(F_\Theta) \nsubseteq W$, at least one variable required by the filter is removed by projection. The filter may then evaluate to error because a required variable is unbound, whereas it may have evaluated to true or false before projection. Hence the equivalence can fail.

For Θ-joins, this means that variables used by later divergence constraints must not be projected away before the corresponding compatibility decision has been evaluated.

S.9.3 Reassociation Conditions

For pure inner joins without intervening projection, Θ-joins can be reassociated by rewriting them as ordinary joins followed by conjunctions of filter predicates. The key condition is that each divergence predicate is evaluated only after all variables it references are in scope.

Let $\Omega_1, \Omega_2, \Omega_3$ be solution-mapping sets, and let Θ be a finite set of divergence constraints over variables drawn from the three operands. Let F_Θ be the conjunction of the corresponding filter predicates. Then, whenever all variables required by F_Θ are in scope after the three-way join,

$$\sigma_{F_\Theta}\big((\Omega_1 \bowtie \Omega_2) \bowtie \Omega_3\big) = \sigma_{F_\Theta}\big(\Omega_1 \bowtie (\Omega_2 \bowtie \Omega_3)\big),$$

because ordinary join is associative and the same conjunction of divergence predicates is evaluated over the same combined mappings.

However, an implementation that evaluates some divergence constraints early must respect variable scope. A constraint involving variables from Ω_1 and Ω_3 cannot be evaluated during a join between Ω_1 and Ω_2, because the variable from Ω_3 is not yet bound. Such constraints must be delayed until both operands contributing the referenced variables have been joined.

S.9.4 Interaction with OPTIONAL, UNION, MINUS, and DIFF

The equivalence in Proposition S.1 is stated for inner joins. Other SPARQL algebra operators require additional care because probabilistic operators are partial and SPARQL filters have three-valued behavior: true, false, and error.

OPTIONAL. For optional patterns, moving a divergence predicate across an optional boundary can change results if variables used by the predicate may remain unbound. If a distribution-valued variable is introduced only by the optional branch, then evaluating `prob:divergenceTest` outside the optional pattern may yield an error on mappings where that branch did not match. Therefore, such rewrites are safe only when all variables referenced by the divergence predicate are guaranteed to be bound.

UNION. For unions, a divergence predicate can be pushed into each branch only if all branches bind the variables required by the predicate with compatible datatypes and domains. If a variable is bound in one branch but not in another, pushing or pulling the predicate across the union can change whether the expression evaluates to false or error.

MINUS and DIFF. For difference-like operators, moving divergence predicates across the operator can change which mappings are removed, especially when variables are unbound or when probabilistic literal interpretation is undefined. Safe rewrites require that the predicate variables are bound before the difference operation and that the predicate does not depend on variables that are only introduced on the right-hand side.

Error propagation. Malformed probabilistic literals, incompatible domains, dimensionality mismatches, or unsupported datatype/operator combinations make probabilistic operator evaluation undefined. In filter contexts, such undefined evaluations produce errors and are handled according to standard SPARQL filter error propagation. Consequently, algebraic rewrites involving probabilistic predicates are valid only when they preserve both variable scope and error behavior.

S.10 Implementation Details and Reproducibility

S.10.1 Apache Jena ARQ Extension

The prototype extends Apache Jena ARQ with probabilistic datatype parsing, probabilistic expression evaluation, and divergence-join execution. Probabilistic RDF literals are parsed into internal value objects before operator evaluation. Datatype-specific backends implement distribution evaluation, transformation, uncertainty propagation, divergence estimation, and serialization back to RDF literals when query results contain distribution-valued outputs.

The extended query processor can be used through the ARQ execution layer and, for endpoint-based experiments, through an Apache Jena Fuseki deployment. This allows probabilistic filters and divergence-based joins to be evaluated inside the SPARQL engine rather than by exporting distribution literals to an external post-processing script.

S.10.2 Datatype Registry

The probabilistic datatype layer is organized as a registry. Each registered datatype provides:

- a parser from lexical forms to internal probabilistic value objects;
- a serializer from internal value objects back to RDF literals;
- a value representation for the corresponding distribution family;
- implementations of supported distribution-valued and deterministic-valued operators;
- divergence or distance routines where comparison operators are supported.

This design separates graph-pattern evaluation from datatype-specific probabilistic computation. Additional probabilistic datatypes can therefore be added by registering the corresponding parser, serializer, value representation, and operator backend, without changing the surrounding SPARQL graph-pattern evaluation machinery.

S.10.3 Artifact Structure

The reproducibility package is organized as follows:

- `src/`: ProbSPARQL implementation, Apache Jena ARQ extensions, datatype registry, and datatype-specific operator backends.
- `data/`: generated circular-factory benchmark graphs and configuration files for dataset generation.
- `queries/`: ProbSPARQL query workloads for R1–R4 and the multi-dimensional GMM experiments.
- `scripts/`: scripts for dataset generation, benchmark execution, and result aggregation.
- `results/`: raw benchmark outputs and summarized experimental results, where available.
- `README.md`: artifact setup instructions, software requirements, execution workflow, and description of the expected outputs.

The benchmark scripts fix random seeds for generated distributions and query workloads where randomness is involved. This makes the generated RDF graphs, candidate-pair workloads, and reported aggregate metrics reproducible from the artifact configuration.

S.10.4 Reproduction Workflow

The experiments can be reproduced using the following workflow.

1. **Build the prototype.** Install the required Java environment and build the ProbSPARQL extension together with the datatype backends.
2. **Generate or load benchmark data.** Use the provided dataset-generation scripts, or load the generated RDF benchmark graphs from the artifact package. The generated graphs follow the circular-factory schema described in Section S.7.
3. **Start the query engine.** Run the extended Apache Jena ARQ processor directly or start the configured Apache Jena Fuseki endpoint with the ProbSPARQL extension and registered probabilistic datatypes.
4. **Execute query workloads.** Run the R1–R4 query workloads from Section S.6 and, where applicable.
5. **Aggregate results.** Use the provided aggregation scripts to compute median latency, standard deviation, precision, recall, F1 score, and mean absolute error where applicable. Unless otherwise stated, latency values are aggregated as the median of 10 measured runs after 3 warm-up iterations.

The software environment used for the main experiments is reported in Section S.7.4. Reproduction on different hardware may change absolute latency values, but should preserve the relative comparison between deterministic baselines, in-engine probabilistic evaluation, and divergence-decision strategies.